

```
Terminal
x
codio@cleverlogo-eventcricket:~/workspace$ cd learning-poetry
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry remove fire
Updating dependencies
Resolving dependencies... (0.1s)

Writing lock file

Package operations: 0 installs, 0 updates, 2 removals

  • Removing fire (0.5.0)
  • Removing termcolor (1.1.0)
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry show
attrs                22.2.0 Classes Without Boilerplate
importlib-metadata   4.8.3  Read metadata from Python packages
more-itertools       8.14.0 More routines for operating on iterables, beyond itertools
packaging            21.3   Core utilities for Python packages
pluggy              0.13.1 plugin and hook calling mechanisms for python
py                   1.11.0 library with cross-python path, ini-parsing, io, code, log facilities
pyparsing            3.0.7  Python parsing module
pytest               5.4.3  pytest: simple powerful testing with Python
python-dateutil       2.8.2  Extensions to the standard Python datetime module
six                  1.16.0 Python 2 and 3 compatibility utilities
typing-extensions    4.1.1  Backported and Experimental Type Hints for Python 3.6+
wcwidth              0.2.6  Measures the displayed width of unicode strings in a terminal
zipp                  3.6.0  Backport of pathlib-compatible object wrapper for zip files
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry remove python-dateutil
Updating dependencies
Resolving dependencies... (0.1s)

Writing lock file

Package operations: 0 installs, 0 updates, 2 removals

  • Removing python-dateutil (2.8.2)
  • Removing six (1.16.0)
codio@cleverlogo-eventcricket:~/workspace/learning-poetry$ poetry show --tree
pytest 5.4.3 pytest: simple powerful testing with Python
├── atomicwrites >=1.0
├── attrs >=17.4.0
├── colorama *
├── importlib-metadata >=0.12
│   ├── typing-extensions >=3.6.4
│   └── zipp >=0.5
├── more-itertools >=4.0.0
├── packaging *
│   ├── pyparsing >=2.0.2,<3.0.5 || >3.0.5
│   └── pluggy >=0.12,<1.0
│       ├── importlib-metadata >=0.12
│       │   ├── typing-extensions >=3.6.4
│       │   └── zipp >=0.5
└── py >=1.5.0
```

Removing Dependencies

Remove a Dependency

Start by changing into the `learning-poetry` directory.

```
cd learning-poetry
```

If `add` is used to add a dependency to a project, use `remove` with Poetry to remove a dependency.

```
poetry remove fire
```

Now, display the dependencies in list form.

```
poetry show
```

Compare the list here with the tree structure above. The `fire` package has `termcolor` listed as a dependency. Removing `fire` not only removes the package but also its dependencies (assuming they are not needed by another package). With `pip`, you would have to manually remove these dependencies.

Try this variation:

- Remove the `python-dateutil` package.

```
poetry remove python-dateutil
```

Now view the dependencies in the tree form; `python-dateutil` and its dependencies should no longer be listed.

```
poetry show --tree
```

Virtual Environments

You may have noticed that the acts of adding and removing dependencies were done without mention of a virtual environment. However, take a look at the first line of terminal output when adding a dependency for the very first time. **Note**, the exact name of the virtual environment may be different than the one below.

```
Creating virtualenv learning-poetry-LfPaow_U-py3.6 in /home/codio/.cache/pypoetry/v...
```

Poetry automatically creates and activates a virtual environment for you. During subsequent package installations, you will no longer see the message about creating a virtual environment. It still happens, but Poetry does not write a message to the terminal.

Use the following command to list all of the virtual environments associated with `learning-poetry`.

```
poetry env list
```

Just the simple act of changing into the `learning-poetry` directory activates the associated virtual environment. You do not have to worry about creating, activating, or deactivating a virtual environment with Poetry. However, you do need to be in the right directory.

```
learning-poetry-LfPaow_U-py3.6 (Activated)
```

Reading Question

Fill in the blanks below.

- Manually remove dependencies - `pip`
- Automatically remove dependencies - `Poetry`
- Uses the `add` command to add a dependency - `Poetry`
- Uses the `install` command to add a dependency - `pip`
- Can display a list of installed dependencies - `Poetry` and `pip`

The correct answers are:

- Manually remove dependencies - **pip**
- Automatically remove dependencies - **Poetry**
- Uses the `add` command to add a dependency - **Poetry**
- Uses the `install` command to add a dependency - **pip**
- Can display a list of installed dependencies - **Poetry** and **pip**

Poetry automates tasks like dependency removal, while you have to do this manually in `pip`. Poetry uses `add` to add a dependency, while `pip` uses `install`. Both `pip` and Poetry can display a list of installed dependencies.

Check It!

Next